CHAPTER 1-INTRODUCTION TO DATABASE

1.1 Objective

- Introduction to Database
- Understand types of Database and its architecture
- Understand Database server and its instances
- Understand Normalization
- Understand procedure of 'Connecting to Server from Client'
- Understand SQL Data Types and concepts on DDL, DML, Constraints, DRL including aggregate functions, joins and sub-queries

1.2 Content

1.2.1 Introduction to Database

Data : It is unprocessed raw facts which has no meaning. For example, 300 is data.

Information : It is processed data which helps in decision making and planning. When we add meaning to data, it becomes information.
For example, 300 is the total strength of associates present today at Ahmadabad ILP center .

Information = Data + context , in which data is 300 and context is ' total strength of associates present today at Ahmadabad ILP center'.

We need to manage data because of following problems with data management:

- Redundancy
Data existing at multiple places
- Inconsistency
Stale data, corrupted data
- Unavailability
Data doesn't exist

Quality information needs data management. Data management means managing creation, reading, update & deletion of data.

Approaches to data management:

- File
- Database

Advantage of File System :
-Cost effective for smaller data size
-Data capturing is simple
Disadvantage of File System :

-Unable to manage complex data
- Data searching may be difficult
- Large data can not be managed

Database System :

A database is an organized collection of data. Using database, we can input, store, retrieve and manage the large data. The data are typically organized to find relevant information. For example, modeling the availability of rooms in hotels in a way that supports finding a hotel with vacancies.

-It has a Database Management System (DBMS) - collection of interrelated data and programs to access the data.
-It has a data model – the manner in which data is stored.
-Uses a programming language to design and manage data - (Data definition language-DDL, Data Manipulation language-DML)

Advantage of Database System :

-It organize, store and retrieve large amounts of data
-Data quality is ensured
-Data structure can be maintained

Disadvantage of Database System :

-Need a DBMS
-Start up effort required
-Training effort required

Database is collection of tables and each table contains rows and columns .

For example, below mentioned 'Customers' table contains 3 rows and 3 columns (CustomerID, CustomerName, City)

Customers Table

CustomerID	CustomerName	City
1	Ram	Delhi
2	Sita	Bangalore
3	Shyam	Hyderabad

1.2.2 Types of Database and its architecture

Types of databases include analytic, management, operational, flat-file and hierarchical databases. Other databases include end-user, network, distributed and rational databases. The difference in the databases is based on how it works and what it is used for.

There are many types of databases that vary by function and data model.

<u>One way to classify different types of databases is by their function:</u>

- **Analytical databases,** sometimes called On-line Analytical Processes (OLAP) are designed to assist in decision-making. Typically, an OLAP database contains statistical information that is read by users, but not modified.
- **Operational databases**, also known as On-line Transaction Processing (OLTP) can be manipulated by users. Records can be added, deleted or modified by users with the proper level of access.

<u>The second way to classify different types of databases is by their 'data model'.</u>

A data model describes the structure of data and how it is accessed. There are several common data models:

- Flat-file. Flat file databases most closely resemble paper files and file cabinets. Flat file databases, while simple to create and access, contain lots of redundant information. These redundancies slow down the search process, making the flat-file type of database inefficient.

- Hierarchical databases relate tables in a parent/child format. Although this reduces redundancy and increases organization, it is still a limiting way to handle data. The fact that child tables at the same level don't have links to each other makes searching problematic.

- Network databases contain simple data that exist with or without links to other data. The database is separated into records, which may be sliced into columns or fields.

- Relational. Relational databases link tables with common "key" fields allowing for sophisticated relationships between tables. Table links can be "indexed" or stored for future use, resulting in quicker searches in this type of database.

- Object-oriented. Object oriented databases are structurally similar to relational databases. While most other types of databases are designed to hold text and numbers, object-oriented databases are designed to store audio, video and other mixed media objects.

A database management system (DBMS) is an aggregate of data, hardware, software, and users that helps an enterprise manage its operational data. DBMS is responsible for maintaining the integrity and security of stored data, and for recovering information if the system fails. In case of DBMS, we can not establish relation between tables . It can follow network, hierarchical or other data model. In DBMS, we need to open each and every table whenever we want to use it.

<u>Advantages of DBMS :</u>

- Reduced data redundancy

- ⇧ Greater data integrity and consistency
- ⇧ Improved data security
- ⇧ Reduced data entry, storage and retrieval cost
- ⇧ Improved data access to users through use of host and query language

Disadvantages of DBMS :

- ⇧ Database systems are complex, difficult and time consuming to design
- ⇧ Damage to databases affects virtually to all applications programs
- ⇧ Substantial hardware and software start up costs
- ⇧ Initial training is required for all programmers and users

RDBMS stands for Relational Database Management System. In case of RDBMS, we can establish relation between tables and it is based on the relational model. In RDBMS, if we open database, the whole tables can be accessible.
It is basis for SQL, and for all modern database systems such as MS SQL Server, IBM DB2, Oracle, MySQL, and Microsoft Access. RDBMS allows the data to be queried based on any column in any table and no need to create an index in order to query data so relational data is easier to query than hierarchical.

Structure Of DataBase

When we use a database, we are not usually concerned with where each piece of data is stored, or what size it is. We just want to be sure that when you refer to a name, for example, the correct value is returned.

- ⇧ Physical database structure

The physical database describes how the structures in the logical database and the search paths between them are implemented. The term database means the logical database, unless indicated otherwise.
One or more data files, two are more redo log files, and one or more control files are components of logical database structure of Oracle database.

It specifies the physical configuration of the database on the storage media. At a physical level, the data is stored in data files on disk . The data in the data files is stored in operating system blocks. It is the detailed design of a system that includes modules & the database's hardware & software specifications of the system.

One characteristic of an RDBMS is the independence of logical data structures such as tables, views, and indexes from physical storage structures. Because physical and logical structures are separate, you can manage physical storage of data without affecting access to logical structures. For example, renaming a database file does not rename the tables stored in it.

•Data files and temp files

A data file is a physical file on disk that was created by Oracle Database and contains data structures such as tables and indexes. A temp file is a data file that belongs to a temporary table space. The data is written to these files in an Oracle proprietary format that cannot be read by other programs.

•Control files

A control file is a root file that tracks the physical components of the database.

•Online redo log files

The online redo log is a set of files containing records of changes made to data.

A database instance is a set of memory structures that manage database files.

⇧ Logical database structure

The logical database is the structure of the data and the relationships between different pieces of information. There is no information about how these structures and relations are implemented. Tablespaces and database's schema objects are components of logical database structure of Oracle database.

Once the relationships and dependencies amongst the various pieces of information have been determined, it is possible to arrange the data into a logical structure which can then be mapped into the storage objects supported by the database management system. In the case of relational databases the storage objects are tables which store data in rows and columns.
Oracle Database allocates logical space for all data in the database. The logical units of database space allocation are data blocks, extents, segments, and table spaces.

At the finest level of granularity, Oracle Database stores data in data blocks. One logical data block corresponds to a specific number of bytes of physical disk space, for example, 2 KB. Data blocks are the smallest units of storage that Oracle Database can use or allocate.

An extent is a set of logically contiguous data blocks allocated for storing a specific type of information.

A segment is a set of extents allocated for a specific database object, such as a table. For example, the data for the employees table is stored in its own data segment, whereas each index for employees is stored in its own index segment. Every database object that consumes storage consists of a single segment.

1.2.3 Database server and its instances

A database server is a computer program that provides database services to other computer programs or computers, as defined by the client–server model. The term may also refer to a computer dedicated to running such a program.
Most of the Database servers works with the base of Query Language. Each Database understands its query language and converts it to Server readable form and executes it to retrieve the results.

Some examples of proprietary database servers are Oracle, DB2, Informix, and Microsoft SQL Server. Every server uses its own query logic and structure. The SQL query language is more or less the same in all relational database servers

An instance of the Database Engine is a copy of the sqlservr.exe executable that runs as an operating system service. Each instance manages several system databases and one or more user databases. Each computer can run multiple instances of the Database Engine. Applications connect to the instance in order to perform work in a database managed by the instance.

A database instance is a set of memory structures that manage database files. A database is a set of physical files on disk created by the CREATE DATABASE statement. The instance manages its associated data and serves the users of the database.

Every running Oracle database is associated with at least one Oracle database instance. Because an instance exists in memory and a database exists on disk, an instance can exist without a database and a database can exist without an instance.

Database Instance Structure
When an instance is started, Oracle Database allocates a memory area called the system global area (SGA) and starts one or more background processes. The SGA serves various purposes, including the following:

•Maintaining internal data structures that are accessed by many processes and threads concurrently

•Caching data blocks read from disk

•Buffering redo data before writing it to the online redo log files

•Storing SQL execution plans

The SGA is shared by the Oracle processes, which include server processes and background processes, running on a single computer. The way in which Oracle processes are associated with the SGA varies according to operating system.

A database instance includes background processes. Server processes, and the process memory allocated in these processes, also exist in the instance. The instance continues to function when server processes terminate.

To execute any query, we have to start oracle instance using STARTUP command.

1.2.4 Normalization

Normalization is the process of organizing the fields and tables of a relational database to minimize redundancy and dependency.

To free the database from Insertion,Update and Deletion anomalies.

Normalization is used for mainly two purpose,

1. Eliminating redundant(useless) data.
2. Ensuring data dependencies make sense i.e data is logically stored.
3. Without Normalization,it becomes difficult to handle and update the database, without facing data loss.
4. Insertion, Updation and Deletion anomalies are very frequent.

A table which is not in Normalized form:

	S_ID	S_NAME	S_ADDRESS	SUBJECT_OPTED
	NUMBER (38)	VARCHAR2 (255)	VARCHAR2 (255)	VARCHAR2 (255)
1	401	nandish	bangalore	biology
2	402	sujit	hubli	physics
3	403	varun	dharwad	maths
4	404	narendra	hyderabad	maths
5	405	nandish	bangalore	physics

Types of Anomolies

- **Updation Anomaly :**

  To update address of a student who occurs twice or more than twice in a table, we will have to update S_Address column in all the rows, else data will become inconsistent.

  Table showing Updation Anomaly:

	S_ID	S_NAME	S_ADDRESS	SUBJECT_OPTED
	NUMBER (38)	VARCHAR2 (255)	VARCHAR2 (255)	VARCHAR2 (255)
1	401	nandish	bangalore	biology
2	402	sujit	hubli	physics
3	403	varun	dharwad	maths
4	404	narendra	hyderabad	maths
5	405	nandish	bangalore	physics

- **Insertion Anomaly :**

  Suppose for a new admission, we have a Student id(S_id), name and address of a student but if student has not opted for any subjects yet then we have to insert NULL there, leading to Insertion anomaly.

  Table showing Insertion Anomaly:

	S_ID	S_NAME	S_ADDRESS	SUBJECT_OPTED
	NUMBER (38)	VARCHAR2 (255)	VARCHAR2 (255)	VARCHAR2 (255)
1	401	nandish	bangalore	biology
2	402	sujit	hubli	physics
3	403	varun	dharwad	maths
4	404	narendra	hyderabad	maths
5	405	nandish	bangalore	physics
6	406	Anubhav	bangalore	NULL

- **Deletion Anomaly :**

  If (S_id) 403 has only one subject and temporarily he drops it, when we delete that row entire student record will be deleted along with it.

  Table showing Deletion Anomaly:

	S_ID	S_NAME	S_ADDRESS	SUBJECT_OPTED
	NUMBER (38)	VARCHAR2 (255)	VARCHAR2 (255)	VARCHAR2 (255)
1	401	nandish	bangalore	biology
2	402	sujit	hubli	physics
3	403	varun	dharwad	maths
4	404	narendra	hyderabad	maths
5	405	nandish	bangalore	physics

Normal Forms:

- **First Normal Form (1NF)**

  A row of data cannot contain repeating group of data

  Each row of data must have a unique identifier i.e Primary key.

  A table which is not in First normal form:

	S_ID	S_NAME	SUBJECT
	NUMBER (38)	VARCHAR2 (255)	VARCHAR2 (255)
1	401	nandish	biology
2	402	sujit	physics
3	403	varun	maths
4	404	narendra	maths
5	401	nandish	physics

Student name 'Nandish' is used twice in the table and subject maths is also repeated.

To reduce above table to First Normal form break the table into two different tables.

Student Table:

	S_ID	S_NAME
	NUMBER (38)	VARCHAR2 (255)
1	401	nandish
2	402	sujit
3	403	varun
4	404	narendra

Subject Table:

	SUBJECT_ID	STUDENT_ID	SUBJECT
	NUMBER (38)	NUMBER (38)	VARCHAR2 (255)
1	1	401	biology
2	2	402	physics
3	3	403	maths
4	3	404	maths
5	2	401	physics

- ⇧ In Student table concatenation of subject_id and student_id is the Primary key.
- ⇧ Now both the Student table and Subject table are normalized to first normal form.

⇧ **Second Normal Form (2NF)**

Should meet all the needs of First Normal Form. There must not be any partial dependency.

A table which is not in Second normal form:

	CUSTOMER_ID	CUSTOMER_NAME	ORDER_ID	ORDER_NAME	SALE_DETAIL
	NUMBER (38)	VARCHAR2 (20)	NUMBER (38)	VARCHAR2 (10)	VARCHAR2 (10)
1	101	Anubhav	10	order1	sale1
2	101	Anubhav	11	order2	sale2
3	102	Dilip	12	order3	sale3
4	103	Kiran	13	order4	sale4

Customer_Name is only dependent on customer_id, Order_name is dependent on Order_id and there is no link between sale_detail and Customer_name.

To reduce above table to Second Normal form break the table into three different tables

Customer_Detail Table :

	CUSTOMER_ID	ORDER_ID	SALE_DETAIL
	NUMBER (4)	NUMBER (4)	VARCHAR2 (10)
1	101	10	Sale1
2	101	11	Sale2
3	102	12	Sale3
4	103	13	Sale4

Order_Detail Table :

	CUSTOMER_ID	CUSTOMER_NAME
	NUMBER (38)	VARCHAR2 (20)
1	101	Anubhav
2	102	Dilip
3	103	Kiran

Sale_Detail Table :

	ORDER_ID	ORDER_NAME
	NUMBER (4)	VARCHAR2 (10)
1	10	Order1
2	11	Order2
3	12	Order3
4	13	Order4

- **Third Normal Form (3NF)**

  Every non-prime attribute of table must be dependent on primary key.The transitive functional dependency should be removed from the table.

Student_id Student_name DOB Street City State Zip

Street, city and state depends upon Zip. The dependency between zip and other fields is called transitive dependency. To apply 3NF, move the street, city and state to new table,with Zip as primary key.

Student_id Student_name DOB Zip

Zip Street City State

BCNF(Boyce and Codd Normal Form)

- It is a higher version of the Third Normal Form.
- This form deals with certain type of anomalies that is not handled by 3NF.
- A 3NF table which does not have multiple overlapping candidate keys is said to be in BCNF.

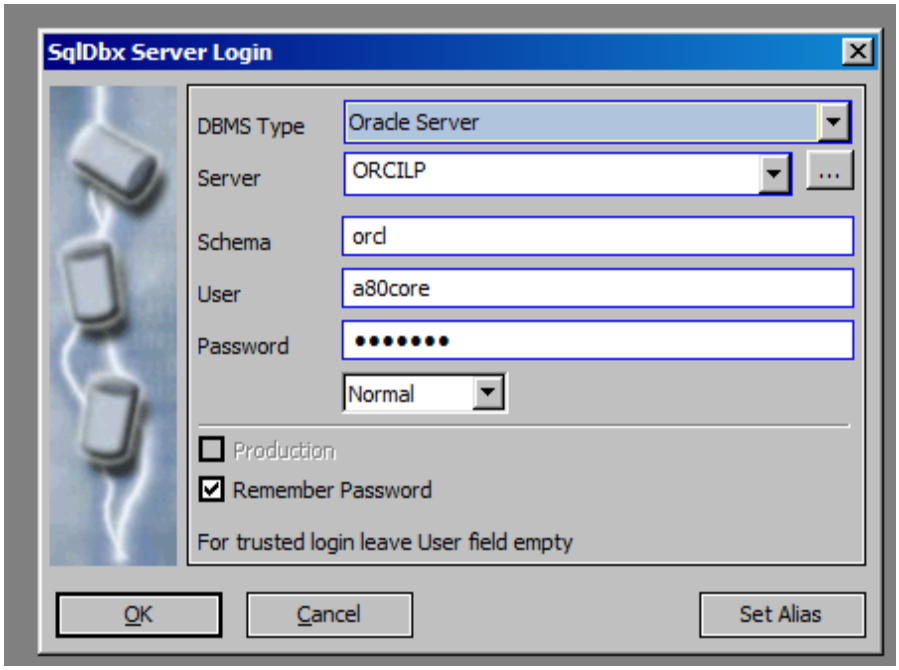
1.2.5 Connecting to Server from Client

We have many ways to connect to database server through client tool .

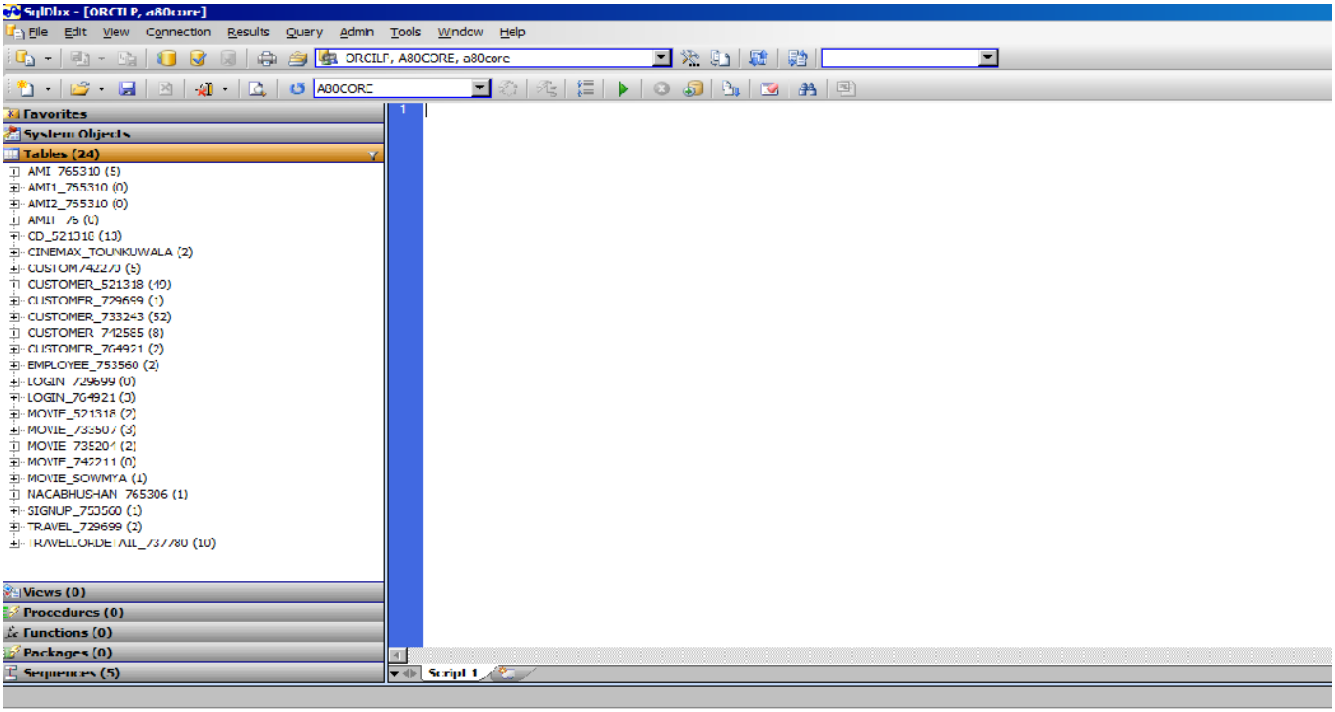
- SqlDbxPersonal tool (SqlDbx)
- Putty (Unix system)

SqlDbxPersonal tool (SqlDbx)

When we start SqlDbx application, it automatically shows Server Login dialog in which we select correct Server type, enter Server name, schema, User and Password and click button OK as shown below .



If server found and client software installed correctly new SQL Editor window will open as shown below.



Putty (Unix system)

We have to make sure that in our system we have putty installed . Go to cmd from start button and reach to that path where putty is installed. Then type below command to open putty as shown in below diagrams :

putty IPAddress
and then enter key .

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation.  All rights reserved.

C:\Users\521318>cd desktop

C:\Users\521318\Desktop>putty 172.26.132.40

C:\Users\521318\Desktop>
```

It will open putty login screen and will ask for id and password as shown in below diagram .

```
172.26.132.40 - PuTTY
login as:
```

Once Id and password are accepted , we will get shell command as shown in below diagrams :

```
172.26.132.40 - PuTTY
login as: 521318
521318@172.26.132.40's password:
Last login: Tue Jun 11 11:49:27 2013 from 10.26.197.63

*****************************************
*****************************************

             Available-HardDisk
              125G        38%

*****************************************
*****************************************

-bash-3.2$
```

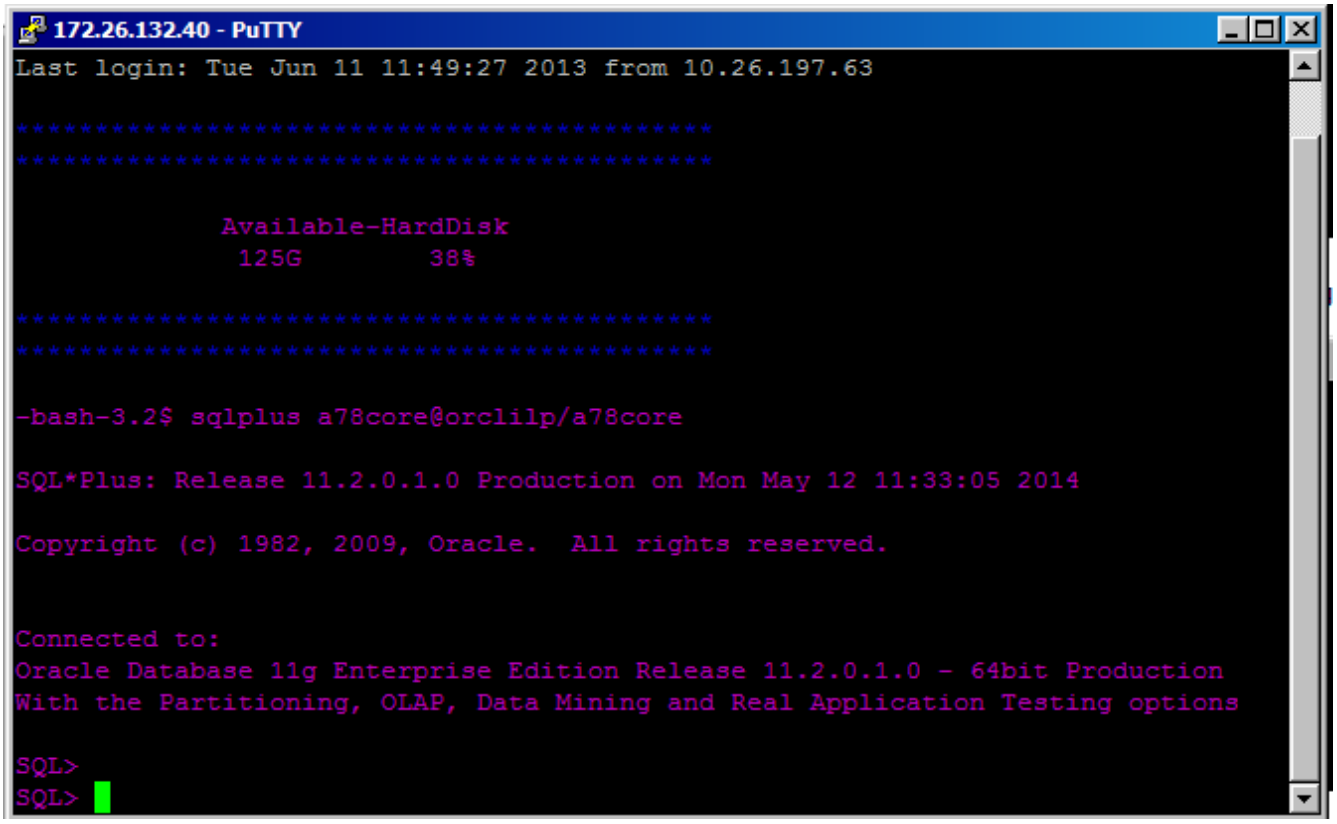
At shell command, we have to type below command for login into database :

sqlplus DbId/DbPassword

and then enter key . We will get below sql prompt where we can write our query and execute it as shown below :

sql> insert into Employee (100,'Ram');

PRESS ENTER KEY FOR QUERY EXECUTION.



1.2.6 Data Types and SQL Concepts

A data type defines what kind of value a column can contain. Database has many data types as shown below :

Float, Data and Time, Number, String, Boolean, BLOB, CLOB etc..

Each column in a database table is required to have a name and a data type. SQL developers have to decide what types of data will be stored inside each and every table column when creating a SQL table.

However, different databases offer different choices for the data type definition.
The following table shows some of the common names of data types between the various database platforms:

Data type	**SQLServer**	**Oracle**	**MySQL**
boolean	Bit	Byte	N/A

integer	Int	Number	Int Integer
float	Float Real	Number	Float
string (fixed)	Char	Char	Char
string (variable)	Varchar	Varchar Varchar2	Varchar

The following table lists the general data types in SQL:

Data type	Description
DATE	Stores year, month, and day values
TIME	Stores hour, minute, and second values
BLOB	A binary string made up of bytes with no associated code page. This data type can store binary data larger than VARBINARY (32K limit). This data type is good for storing image, voice, graphical, and other types of business or application-specific data.
CLOB	A character string made up of single-byte characters with an associated code page. This data type is appropriate for storing text-oriented information where the amount of information can grow beyond the limits of a regular VARCHAR data type (upper limit of 32K bytes). Code page conversion of the information is supported.

SQL Concepts

SQL is a standard language for accessing and manipulating databases. It stands for Structured Query Language. Users interact with database systems through query languages.

DDL : It is Data Definition Language which consist of 'CREATE, DROP, TRUNCATE, ALTER' command.

DML : It is Data Manipulation Language which consist of 'INSERT, DELETE, UPDATE' command.

DRL : It is Data Retrival Language which consist of 'SELECT' command.

CREATE TABLE Statement

The CREATE TABLE statement is used to create a table in a database. Tables are organized into rows and columns; and each table must have a name.

Syntax

CREATE TABLE *table_name*
(
column_name1 data_type(*size*),
column_name2 data_type(*size*),

column_name3 data_type(*size*),
....
);

The column_name parameters specify the names of the columns of the table. The data_type parameter specifies what type of data the column can hold (e.g. varchar, integer, date, etc.). The size parameter specifies the maximum length of the column of the table.

Try it yourself :

Now we want to create a table called "Persons" that contains five columns: PersonID, LastName, FirstName, Address, and City.
We use the following CREATE TABLE statement:

```
CREATE TABLE Persons
(
PersonID int,
LastName varchar(255),
FirstName varchar(255),
Address varchar(255),
City varchar(255)
);
```

DROP TABLE Statement

The DROP TABLE statement is used to delete a table.
DROP TABLE table_name

TRUNCATE TABLE Statement

What if we only want to delete the data inside the table, and not the table itself?
Then, use the TRUNCATE TABLE statement:

TRUNCATE TABLE table_name

The ALTER TABLE Statement

The ALTER TABLE statement is used to add, delete, or modify columns in an existing table.

Syntax

To add a column in a table, use the following syntax:

ALTER TABLE table_name
ADD column_name datatype

To delete a column in a table, use the following syntax (notice that some database systems don't allow deleting a column):

ALTER TABLE table_name
DROP COLUMN column_name

Try it yourself :

Look at the "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

Now we want to add a column named "DateOfBirth" in the "Persons" table.
We use the following SQL statement:

ALTER TABLE Persons
ADD DateOfBirth date

SELECT Statement

The SELECT statement is used to select data from a database.

Syntax

SELECT column_name,column_name
 FROM table_name;

and

 SELECT * FROM table_name;

Try it yourself : SELECT CustomerName,City FROM Customers;

WHERE Clause

The WHERE clause is used to extract only those records that fulfill a specified criterion.

Syntax

SELECT column_name,column_name
 FROM table_name
 WHERE column_name operator value;

Try it yourself :SELECT * FROM Customers WHERE Country='Mexico';

AND & OR Operators

The AND operator displays a record if both the first condition AND the second condition are true.

The OR operator displays a record if either the first condition OR the second condition is true.

```
SELECT * FROM Customers
 WHERE Country='Germany' AND City='Berlin';
```

```
SELECT * FROM Customers
 WHERE City='Berlin' OR City='München';
```

ORDER BY Keyword

The ORDER BY keyword is used to sort the result-set by one or more columns.

The ORDER BY keyword sorts the records in ascending order by default. To sort the records in a descending order, you can use the DESC keyword.

Syntax

```
SELECT column_name,column_name
 FROM table_name
 ORDER BY column_name,column_name ASC|DESC;
```

```
SELECT * FROM Customers
 ORDER BY Country;
```

```
SELECT * FROM Customers
 ORDER BY Country DESC;
```

INSERT Statement

The INSERT INTO statement is used to insert new records in a table.

It is possible to write the INSERT INTO statement in two forms.

The first form does not specify the column names where the data will be inserted, only their values:

```
INSERT INTO table_name
 VALUES (value1,value2,value3,...);
```

The second form specifies both the column names and the values to be inserted:

```
INSERT INTO table_name (column1,column2,column3,...)
 VALUES (value1,value2,value3,...);
```

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
 VALUES ('Cardinal','Tom B. Erichsen','Skagen 21','Stavanger','4006','Norway');
```

UPDATE Statement

The UPDATE statement is used to update existing records in a table.

```
UPDATE table_name
 SET column1=value1,column2=value2,...
 WHERE some_column=some_value;
```

```
UPDATE Customers
 SET ContactName='Alfred Schmidt', City='Hamburg'
 WHERE CustomerName='Alfreds Futterkiste';
```

DELETE Statement

The delete statement is used to delete existing records in a table.

```
DELETE FROM table_name
 WHERE some_column=some_value;
```

```
DELETE FROM Customers
 WHERE CustomerName='Alfreds Futterkiste' AND ContactName='Maria Anders';
```

Delete All Data

It is possible to delete all rows in a table without deleting the table. This means that the table structure, attributes, and indexes will be intact:

```
DELETE FROM table_name;
```

The DELETE statement is used to delete records in a table.

LIKE Operator

The LIKE operator is used to search for a specified pattern in a column.

Syntax

```
SELECT column_name(s)
 FROM table_name
 WHERE column_name LIKE pattern;
```

The following SQL statement selects all customers with a City starting with the letter "s":

```
SELECT * FROM Customers
 WHERE City LIKE 's%';
```

The "%" sign is used to define wild cards (missing letters) both before and after the pattern.

The following SQL statement selects all customers with a City ending with the letter "s":

```
SELECT * FROM Customers
 WHERE City LIKE '%s';
```

IN Operator

The IN operator allows you to specify multiple values in a WHERE clause.

Syntax

```
SELECT column_name(s)
 FROM table_name
 WHERE column_name IN (value1,value2,...);
```

The following SQL statement selects all customers with a City of "Paris" or "London":

```
SELECT * FROM Customers
 WHERE City IN ('Paris','London');
```

BETWEEN Operator

The BETWEEN operator is used to select values within a range.The values can be numbers, text, or dates.

Syntax

```
 SELECT column_name(s)
 FROM table_name
 WHERE column_name BETWEEN value1 AND value2;
```

```
SELECT * FROM Products WHERE Price BETWEEN 10 AND 20;
```

SQL Joins

An SQL JOIN clause is used to combine rows from two or more tables, based on a common field between them.

The most common type of join is: SQL **INNER JOIN** (simple join). An SQL INNER JOIN return all rows from multiple tables where the join condition is met.

INNER JOIN or NORMAL JOIN :

Returns all rows when there is at least one match in BOTH tables.
The INNER JOIN keyword selects all rows from both tables as long as there is a match between the columns in both tables.

Syntax

SELECT *column_name(s)*
FROM *table1*
INNER JOIN *table2*
ON *table1.column_name=table2.column_name*;
or:
SELECT *column_name(s)*
FROM *table1*
JOIN *table2*
ON *table1.column_name=table2.column_name*;
Note : *INNER JOIN is the same as JOIN.*

Let's look at a selection from the "Orders" table:

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

Then, have a look at a selection from the "Customers" table:

CustomerID	CustomerName	Country
1	Alfreds Futterkiste	Germany
2	Ana Trujillo Emparedados y helados	Mexico
3	Antonio Moreno Taquería	Mexico

Notice that the "CustomerID" column in the "Orders" table refers to the customer in the "Customers" table. The relationship between the two tables above is the "CustomerID". Then, if we run the following SQL statement (that contains an **INNER JOIN or NORMAL JOIN**):

Try it yourself :

SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
INNER JOIN Customers
ON Orders.CustomerID=Customers.CustomerID;

Result of Query :

OrderID	CustomerName	OrderDate
10308	Ana Trujillo Emparedados y helados	1996-09-18

LEFT JOIN:

Return all rows from the left table, and the matched rows from the right table. The LEFT JOIN keyword returns all rows from the left table (table1), with the matching rows in the right table (table2).

The result is NULL in the right side when there is no match.

Syntax

SELECT *column_name(s)*
FROM *table1*
LEFT JOIN *table2*
ON *table1.column_name=table2.column_name*;
or:
SELECT *column_name(s)*
FROM *table1*
LEFT OUTER JOIN *table2*
ON *table1.column_name=table2.column_name*;

Note : *In some databases LEFT JOIN is called LEFT OUTER JOIN.*

Try it yourself :

```
SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
LEFT JOIN Customers
ON Orders.CustomerID=Customers.CustomerID;
```

Result of Query :

OrderID	CustomerName	OrderDate
10308	Ana Trujillo Emparedados y helados	1996-09-18
10309	Null	1996-09-19
10310	Null	1996-09-20

RIGHT JOIN:

Return all rows from the right table, and the matched rows from the left table. The RIGHT JOIN keyword returns all rows from the right table (table2), with the matching rows in the left table (table1). The result is NULL in the left side when there is no match.

Syntax

SELECT *column_name(s)*
FROM *table1*
RIGHT JOIN *table2*
ON *table1.column_name=table2.column_name*;

or:

SELECT *column_name(s)*
FROM *table1*
RIGHT OUTER JOIN *table2*
ON *table1.column_name=table2.column_name*;

Note: *In some databases RIGHT JOIN is called RIGHT OUTER JOIN.*

Try it yourself :

SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
RIGHT JOIN Customers
ON Orders.CustomerID=Customers.CustomerID;

Result of Query :

OrderID	CustomerName	OrderDate
Null	Alfreds Futterkiste	Null
10308	Ana Trujillo Emparedados y helados	09/18/96
Null	Antonio Moreno Taquería	Null

FULL JOIN:

Return all rows from both the tables if there is any match or not. The FULL OUTER JOIN keyword returns all rows from the left table (table1) and from the right table (table2).
The FULL OUTER JOIN keyword combines the result of both LEFT and RIGHT joins.

Syntax
SELECT *column_name(s)*
FROM *table1*
FULL OUTER JOIN *table2*
ON *table1.column_name=table2.column_name*;

Try it yourself :

SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Orders
FULL OUTER JOIN Customers
ON Orders.CustomerID=Customers.CustomerID;

Result of Query :

OrderID	CustomerName	OrderDate

10308	Ana Trujillo Emparedados y helados	1996-09-18
10309	Null	1996-09-19
10310	Null	1996-09-20
Null	Alfreds Futterkiste	Null
Null	Antonio Moreno Taquería	Null

SELF JOIN:

It is a join in which a table is joined with itself, specially when the table has a FOREIGN KEY which references its own PRIMARY KEY. To join a table itself means that each row of the table is combined with itself and with every other row of the table. The self join can be viewed as a join of two copies of the same table.

Syntax
SELECT t1.*column_name, t2. column_name*
FROM *table t1, table t2*
where t1.*column_name = t2.column_name*

Try it yourself :

Consider the following Employee table as follows:

```
CREATE TABLE Employee(
emp_id varchar(5) NOT NULL,
emp_name varchar(20) NULL,
dt_of_join date NULL,
emp_supv varchar(5) NULL,
CONSTRAINT emp_id PRIMARY KEY(emp_id) ,
CONSTRAINT emp_supv FOREIGN KEY(emp_supv)
REFERENCES employee(emp_id)
)
```

We have the following data into the table EMPLOYEE.

EMP_ID	EMP_NAME	DT_OF_JOIN	EMP_SUPV
20051	Vijes Setthi	15-JUN-09	-
20073	Unnath Nayar	09-AUG-10	20051
20064	Rakesh Patel	23-OCT-09	20073
20069	Anant Kumar	03-DEC-08	20051
20055	Vinod Rathor	27-NOV-09	20051
20075	Mukesh Singh	25-JAN-11	20073

The above data shows :

- Unnath Nayar's supervisor is Vijes Setthi
- Anant Kumar and Vinod Rathor can also report to Vijes Setthi.
- Rakesh Patel and Mukesh Singh are under supervison of Unnith Nayar.

Example of SQL SELF JOIN

In the following example we will use the table EMPLOYEE twice and in order to do this we will use the alias of the table.

To get the list of employees and their supervisor the following sql statement has used :

view plain copy to clipboard print ?

```
01. SELECT a.emp_id AS "Emp_ID",a.emp_name AS "Employee Name",
02. b.emp_id AS "Supervisor ID",b.emp_name AS "Supervisor Name"
03. FROM employee a, employee b
04. WHERE a.emp_supv = b.emp_id;
```

Output

Emp_ID	Employee Name	Supervisor ID	Supervisor Name
20055	Vinod Rathor	20051	Vijes Setthi
20069	Anant Kumar	20051	Vijes Setthi
20073	Unnath Nayar	20051	Vijes Setthi
20075	Mukesh Singh	20073	Unnath Nayar
20064	Rakesh Patel	20073	Unnath Nayar

SQL Constraints

SQL constraints are used to specify rules for the data in a table. It helps to ensure the data type and

range that goes into the database entity.
If there is any violation between the constraint and the data action, the action is aborted by the constraint.
Constraints can be specified when the table is created (inside the CREATE TABLE statement) or after the table is created (inside the ALTER TABLE statement).

SQL CREATE TABLE + CONSTRAINT Syntax

CREATE TABLE *table_name*
(
column_name1 data_type(*size*) *constraint_name*,
column_name2 data_type(*size*) *constraint_name*,
column_name3 data_type(*size*) *constraint_name*,
....
);

In SQL, we have the following constraints:

NOT NULL - Indicates that a column cannot store NULL value

UNIQUE - Ensures that each row for a column must have a unique value

PRIMARY KEY - A combination of a NOT NULL and UNIQUE. Ensures that a column (or combination of two or more columns) have an unique identity which helps to find a particular record in a table more easily and quickly.

FOREIGN KEY -

It points to a primary key in another table ensure the referential integrity of the data in one table to match values in another table

CHECK - Ensures that the value in a column meets a specific condition

DEFAULT - Specifies a default value when specified none for this column

NOT NULL Constraint
The NOT NULL constraint enforces a column to NOT accept NULL values.

The NOT NULL constraint enforces a field to always contain a value. This means that you cannot insert a new record, or update a record without adding a value to this field.

The following SQL enforces the "P_Id" column as **PRIMARY KEY a**nd the "LastName" column to **not accept NULL values**:

CREATE TABLE Persons
(
P_Id int **PRIMARY KEY**,
LastName varchar(255) **NOT NULL**,
FirstName varchar(255),

```
Address varchar(255),
City varchar(255)
)
```

UNIQUE Constraint

The UNIQUE constraint uniquely identifies each record in a database table.
The UNIQUE and PRIMARY KEY constraints both provide a guarantee for uniqueness for a column or set of columns. A PRIMARY KEY constraint automatically has a UNIQUE constraint defined on it.

Note : *We may have many UNIQUE constraints per table, but only one PRIMARY KEY constraint per table.*

The following SQL creates a UNIQUE constraint on the "P_Id" column when the "Persons" table is created:

```
CREATE TABLE Persons
(
P_Id int UNIQUE,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

SQL UNIQUE Constraint on ALTER TABLE

To create a UNIQUE constraint on the "P_Id" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons
ADD UNIQUE (P_Id)
```

or

```
ALTER TABLE Persons
ADD CONSTRAINT uc_0 UNIQUE(P_Id)
```

To DROP a UNIQUE Constraint

```
ALTER TABLE Persons
DROP CONSTRAINT  uc_0
```

PRIMARY KEY Constraint

The PRIMARY KEY constraint uniquely identifies each record in a database table.
Primary keys must contain unique values. A primary key column cannot contain NULL values.

Each table should have a primary key, and each table can have only ONE primary key.

SQL PRIMARY KEY Constraint on CREATE TABLE
The following SQL creates a PRIMARY KEY on the "P_Id" column when the "Persons" table is created:

```
CREATE TABLE Persons
(
P_Id int ,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
PRIMARY KEY (P_Id)
)
```

or

```
CREATE TABLE Persons
(
P_Id int PRIMARY KEY,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

SQL PRIMARY KEY Constraint on ALTER TABLE
To create a PRIMARY KEY constraint on the "P_Id" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons
ADD PRIMARY KEY (P_Id)
```

or

```
ALTER TABLE Persons
ADD CONSTRAINT pk_0 PRIMARY KEY(P_Id)
```

To DROP a PRIMARY KEY Constraint

```
ALTER TABLE Persons
DROP CONSTRAINT  pk_0
```

FOREIGN KEY Constraint

A FOREIGN KEY in one table points to a PRIMARY KEY in another table.
Let's illustrate the foreign key with an example. Look at the following two tables:

The "Persons" table:

P_Id	LastName	FirstName	Address	City
1	Hansen	Ola	Timoteivn 10	Sandnes
2	Svendson	Tove	Borgvn 23	Sandnes
3	Pettersen	Kari	Storgt 20	Stavanger

The "Orders" table:

O_Id	OrderNo	P_Id
1	77895	3
2	44678	3
3	22456	2
4	24562	1

Note that the "P_Id" column in the "Orders" table points to the "P_Id" column in the "Persons" table.
The "P_Id" column in the "Persons" table is the PRIMARY KEY in the "Persons" table.
The "P_Id" column in the "Orders" table is a FOREIGN KEY in the "Orders" table.
The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.
The FOREIGN KEY constraint also prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the table it points to.

```
CREATE TABLE Orders
(
O_Id int PRIMARY KEY,
OrderNo int ,
P_Id INT,
CONSTRAINT fk_PerOrders FOREIGN KEY (P_Id)
 REFERENCES Persons(P_Id)
)
```

SQL FOREIGN KEY Constraint on ALTER TABLE

To create a FOREIGN KEY constraint on the "P_Id" column when the "Orders" table is already created, use the following SQL:

```
ALTER TABLE Orders
ADD FOREIGN KEY (P_Id)
REFERENCES Persons(P_Id)
```

or

```
ALTER TABLE Orders
ADD CONSTRAINT fk_0 FOREIGN KEY (P_Id)
REFERENCES Persons(P_Id)
```

To DROP a FOREIGN KEY Constraint

```
ALTER TABLE Persons
DROP CONSTRAINT  fk_0
```

CHECK Constraint

The CHECK constraint is used to limit the value range that can be placed in a column.
If you define a CHECK constraint on a single column it allows only certain values for this column.
If you define a CHECK constraint on a table it can limit the values in certain columns based on values in other columns in the row.

SQL CHECK Constraint on CREATE TABLE

The following SQL creates a CHECK constraint on the "P_Id" column when the "Persons" table is created. The CHECK constraint specifies that the column "P_Id" must only include integers greater than 0.

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
CHECK (P_Id>0)
)
```

or

```
CREATE TABLE Persons
(
P_Id int NOT NULL CHECK (P_Id>0),
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255)
)
```

To allow naming of a CHECK constraint, and for defining a CHECK constraint on multiple columns, use the following SQL syntax:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255),
CONSTRAINT chk_Person CHECK (P_Id>0 AND City='Bangalore')
)
```

SQL CHECK Constraint on ALTER TABLE

To create a CHECK constraint on the "P_Id" column when the table is already created, use the following SQL:

```
ALTER TABLE Persons
ADD CHECK (P_Id>0)
```

To allow naming of a CHECK constraint, and for defining a CHECK constraint on multiple columns, use the following SQL syntax:

```
ALTER TABLE Persons
ADD CONSTRAINT chk_Person CHECK (P_Id>0 AND City='Bangalore')
```

To DROP a CHECK Constraint

To drop a CHECK constraint, use the following SQL:

```
ALTER TABLE Persons
DROP CONSTRAINT chk_Person
```

DEFAULT Constraint
The DEFAULT constraint is used to insert a default value into a column.
The default value will be added to all new records, if no other value is specified.

SQL DEFAULT Constraint on CREATE TABLE
The following SQL creates a DEFAULT constraint on the "City" column when the "Persons" table is created:

```
CREATE TABLE Persons
(
P_Id int NOT NULL,
LastName varchar(255) NOT NULL,
FirstName varchar(255),
Address varchar(255),
City varchar(255) DEFAULT 'Bangalore'
)
```

SQL Functions

SQL has many built-in functions for performing calculations on data.

SQL Aggregate Functions : SQL aggregate functions return a single value, calculated from values in a column. Useful aggregate functions:

AVG() - Returns the average value
COUNT() - Returns the number of rows
MAX() - Returns the largest value

MIN() - Returns the smallest value
SUM() - Returns the sum

AVG() Function

It returns the average value of a numeric column.

Syntax
SELECT AVG(column_name) FROM table_name

Below is a selection from the "Products" table:

ProductID	ProductName	Price
1	Chais	18
2	Chang	19
3	Aniseed Syrup	10
4	Chef Anton's Cajun Seasoning	21.35
5	Chef Anton's Gumbo Mix	25

SQL AVG() Example
The following SQL statement gets the average value of the "Price" column from the "Products" table:

Try it yourself and verify the output:
SELECT AVG(Price) AS PriceAverage FROM Products;
= 18.67

COUNT() function

It returns the number of rows that matches a specified criteria.

Syntax

The COUNT(column_name) function returns the number of values (NULL values will not be counted) of the specified column:

SELECT COUNT(column_name) FROM table_name;

COUNT(*) Syntax

The COUNT(*) function returns the number of records in a table:

SELECT COUNT(*) FROM table_name;

COUNT(DISTINCT column_name) function returns the number of distinct values of the specified column:

Syntax

SELECT COUNT(DISTINCT column_name) FROM table_name;

Try it yourself and verify the output:

Below is a selection from the "Orders" table:

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10265	7	2	1996-07-25	1
10266	87	3	1996-07-26	3
10267	7	4	1996-07-29	1

SELECT COUNT(DISTINCT CustomerID) FROM table_name;
= 2

The MAX() Function

The MAX() function returns the largest value of the selected column.

Syntax
SELECT MAX(column_name) FROM table_name;

Try it yourself and verify the output:

The following SQL statement gets the largest value of the "Price" column from the "Products" table:
Example
SELECT MAX(Price) FROM Products;
= 25

MIN() Function

The MIN() function returns the smallest value of the selected column.

Syntax
SELECT MIN(column_name) FROM table_name;

Try it yourself and verify the output:

The following SQL statement gets the smallest value of the "Price" column from the "Products" table:
Example
SELECT MIN(Price) FROM Products;
= 10

SUM() Function

The SUM() function returns the total sum of a numeric column.

SELECT SUM(column_name) FROM table_name;

Try it yourself and verify the output:

Below is a selection from the "OrderDetails" table:

OrderDetailID	OrderID	ProductID	Quantity
1	10248	11	12
2	10248	42	10
3	10248	72	5
4	10249	14	9
5	10249	51	40

The following SQL statement finds the sum of all the "Quantity" fields for the "OrderDetails" table:

SELECT SUM(Quantity) FROM OrderDetails;
= 76

GROUP BY Statement

The GROUP BY statement is used in conjunction with the aggregate functions to group the result-set by one or more columns.

Syntax
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name;

Try it yourself and verify the output:
SELECT OrderID,SUM(Quantity) FROM OrderDetails GROUP BY OrderID;

Query Result

OrderID	Quantity
10248	27
10249	49

HAVING Clause

The HAVING clause was added to SQL because the WHERE keyword could not be used with aggregate functions.

Syntax
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE column_name operator value
GROUP BY column_name
HAVING aggregate_function(column_name) operator value;

Try it yourself and verify the output:

SELECT OrderID,SUM(Quantity) FROM OrderDetails GROUP BY OrderID HAVING SUM(Quantity) > 40;

Query Result

OrderID	Quantity
10249	49

Subquery

A subquery is a query within a query. In Oracle, you can create subqueries within your SQL statements. These subqueries can reside in the WHERE clause, the FROM clause, or the SELECT clause.

Subquery or Inner query or Nested query is a query in a query. A subquery is usually added in the WHERE Clause of the sql statement. Most of the time, a subquery is used when you know how to search for a value using a SELECT statement, but do not know the exact value in the database.

Subqueries are an alternate way of returning data from multiple tables.

Subqueries can be used with the following sql statements along with the comparision operators like =, <, >, >=, <= etc.
•SELECT
•INSERT
•UPDATE
•DELETE

Try it yourself and verify the output:

Let's look at a selection from the "Orders" table:

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

Then, have a look at a selection from the "Customers" table:

CustomerID	CustomerName	Country
1	Alfreds Futterkiste	Germany
2	Ana Trujillo Emparedados y helados	Mexico
3	Antonio Moreno Taquería	Mexico

Using sub query, I want to fetch those customer who has placed the orders.

Select CustomerID, CustomerName , Country from Customers where CustomerID in (select

CustomerID from Orders)

Result of the Query

CustomerID	CustomerName	Country
2	Ana Trujillo Emparedados y helados	Mexico

1.2.7 Points to remember

1.2.7.1 Database

- ✓ Data : It is unprocessed raw facts
- ✓ Information : It is processed data which helps in decision making and planning.
- ✓ A database is an organized collection of data. Using database, we can input, store, retrieve and manage the large data.

1.2.7.2 Types of Database

- ✓ DBMS : It is an aggregate of data, hardware, software, and users that helps an enterprise manage its operational data. In case of DBMS, we can not establish relation between tables . It can follow network, hierarchical or other data model.
- ✓ RDBMS : It stands for Relational Database Management System. In case of RDBMS, we can establish relation between tables and it is based on the relational model

1.2.7.3 Normalization

- ✓ It is the process of organizing the fields and tables of a relational database to minimize redundancy and dependency.
- ✓ It has three forms : INF , 2 NF and 3 NF

1.2.7.4 Structure Of Database

- ✓ Physical
- ✓ Logical

1.2.7.5 Connecting to Server from Client through

- ✓ Putty
- ✓ SqlDbxPersonal tool (SqlDbx)

1.2.7.6 Data Types

- ✓ Float, Data and Time, Number, String, Boolean, BLOB, CLOB etc..

1.2.7.7 SQL Concepts

- ✓ It is a standard language for accessing and manipulating databases. It stands for Structured Query Language.
- ✓ DDL : CREATE, DROP, TRUNCATE, ALTER
- ✓ DML : INSERT, DELETE, UPDATE
- ✓ DRL : SELECT
- ✓ JOINS : Left , Right, Full Outer and self join
- ✓ Aggregate Function
- ✓ Group by with Having keyword

1.3 <u>Test Your Knowledge</u>

1) Which keyword returns all rows from the right table with the matching rows in the left table and result is NULL in the left side when there is no match?

a. Normal join
b. left join
c. right join
d. full join

2) If we want to delete all rows from the table permanently, which command will be efficient ?

a. Delete
b. Truncate
c. Drop
d. All of the above

3) Which is true?

a. Create table is DML command
b. Create table is DCL command
c. Create table is DDL command
d. none of the able

4) In delete , we can rollback the data.

a. False
b. True

5) Which commnad is not a DML command ?

a. select
b. Insert
c. Delete
d. Update

6) Which SQL statement selects all customers with a Country containing the pattern "land":

a. SELECT * FROM Customers WHERE Country LIKE '%lan%'
b. SELECT * FROM Customers WHERE Country LIKE 'land'
c. SELECT * FROM Customers WHERE Country LIKE '%land%'
d. None of the above

7) Which sql query is correct ?

a. SELECT * FROM Customers WHERE City IN ('Paris','London');
b. SELECT * FROM Customers WHERE City IN ('Paris' and 'London');
c. SELECT * FROM Customers WHERE City IN 'Paris' and 'London' ;
d. a&b

8) In following query 'SELECT * FROM Products WHERE Price BETWEEN 10 AND 20', which price value will not be selected ?

a. 11
b. 10
c. 19
d. none of the above

9) Which aggregare function is correct ?

a. min()
b. max()
c. count()
d. All of the above

10) Whcih query is correct group by query ?

a. SELECT name,count(*) FROM Customers group by name;
b. SELECT name,count(*) FROM Customers WHERE City IN ('Paris') group by name;
c. SELECT count(*) FROM Customers group by count(*);
d. a&b

Answers:

1. [c]
2. [b]
3. [c]
4. [b]
5. [a]
6. [c]
7. [a]
8. [d]
9. [d]
10. [d]